

Examen de la session ordinaire - Durée (1h30)

Nom et prénom : N° Apogée

Exercice 1 (Entourer la bonne réponse : une seule réponse correcte est possible) (5 pts)

1. Dans un programme multithreadé (Pthreads), que se passe-t-il si l'un des threads effectue un appel système `exit(0)` ?

-
- A. Le thread appelant est bloqué jusqu'à ce que les autres threads aient terminé leur exécution
 - B. Libère les mutex détenus par le thread.
 - C. L'appel échoue car le thread n'a pas le droit de terminer le processus père
 - D. Termine tous les threads du processus et le processus lui-même.

2. Pourquoi la fonction `sleep()` peut-elle retourner avant que la durée spécifiée ne soit totalement écoulée ?

-
- A. Parce que le système d'exploitation accélère parfois l'exécution des programmes.
 - B. Parce qu'un signal peut interrompre l'attente du processus.
 - C. La fonction `sleep` ne retourne jamais en avance, c'est une fonction précise en millisecondes.
 - D. Parce qu'un autre thread est rentré dans la section critique

3. Dans le contexte des variables de condition POSIX (`pthread_cond_wait`), pourquoi cet appel doit-il obligatoirement recevoir un mutex verrouillé en paramètre ?

-
- A. Parce que `pthread_cond_wait` détruit automatiquement le mutex à la fin de l'attente.
 - B. Parce qu'une variable de condition ne peut être utilisée qu'avec un seul thread.
 - C. Parce que cela permet de libérer le mutex et de mettre le thread en attente, évitant ainsi les conditions de concurrence.
 - D. C'est une ancienne contrainte de compatibilité qui n'a plus d'effet fonctionnel aujourd'hui.

4. Quelle est la différence entre les signaux standards (ex : `SIGUSR1`) et les signaux Temps Réel (POSIX Real-Time Signals, ex : `SIGRTMIN`) sous Linux ?

-
- A. Les signaux Temps Réel sont toujours plus rapides à traiter que les signaux standards
 - B. Les signaux standards sont mis en file d'attente alors que les signaux Temps Réel ne le sont pas.
 - C. Les signaux Temps Réel sont mis en file d'attente et peuvent transporter des données supplémentaires, contrairement aux signaux standards.
 - D. Les signaux Temps Réel ne peuvent être utilisés que par le noyau Linux

5. Qu'est-ce qu'un signal "pendant" (ou pending signal) sous Linux ?

-
- A. Un signal qui a été envoyé par un processus mais qui a été perdu par le noyau en raison d'une saturation de la mémoire.
 - B. Un signal qui a été généré et délivré, mais dont le gestionnaire (handler) s'est bloqué lors de son exécution.
 - C. Un signal qui a été généré par le noyau ou un autre processus, mais qui est temporairement retenu par le noyau car le processus destinataire l'a masqué (bloqué).
 - D. Un signal qui s'exécute automatiquement en tâche de fond toutes les secondes à l'aide de la fonction `alarm()`.

Exercice 2 (Questions de compréhension : donnez une réponse brève et bien précise) (3 pts)

1. Quand obtient-on un Deadlock (L'interblocage) ? Et comment l'éviter ?

2. Quelle est la différence fondamentale de comportement entre l'appel système `pthread_mutex_lock()` et l'appel `pthread_mutex_trylock()` ?

Exercice 3 : (4 pts)

```
#include <stdio.h>
#include <unistd.h>
#include <sys/wait.h>
int val = 5;
int main() {
    pid_t p1, p2;
    p1 = fork();
    if (p1 == 0) {
        val += 5;
        p2 = fork();
        if (p2 == 0) {
            val *= 2;
            printf("[A] val = %d\n", val);
        } else {
            wait(NULL);
            val += 2;
            printf("[B] val = %d\n", val);
        }
    } else {
        wait(NULL);
        val -= 3;
        printf("[C] val = %d\n", val);
    }
    return 0;
}
```

1. Donnez l'affichage exact et complet produit sur le terminal lors de l'exécution de ce programme.
2. Dessinez ou listez l'arbre généalogique des processus créés par ce programme (qui est le père de qui ?)
Combien de processus au total ont été exécutés ?

Exercice 4 : (8 pts)

Dans le cadre de l'automatisation d'un entrepôt logistique, vous devez développer en langage C une application de supervision sous Linux. Le programme principal (Superviseur) doit orchestrer le travail de 3 processus enfants (Workers) chargés de simuler le traitement de lots de colis. En parallèle, pour assurer un suivi en temps réel, le Superviseur doit lancer un thread de monitoring.

Cahier des charges fonctionnel :

1. **Initialisation** : Le Superviseur met en place un canal de communication via un unique tube anonyme, puis engendre les 3 processus Workers à l'aide de l'appel système `fork()`.
2. **Traitement des Workers (Fils)** : Chaque Worker simule le traitement d'un lot de N colis (via une boucle d'itérations espacées par `sleep`). A chaque colis traité, le Worker écrit un entier de valeur 1 dans le tube pour notifier sa progression. Une fois son lot terminé, chaque fils ferme ses descripteurs et quitte proprement via `exit(0)`.
3. **Suivi par le Superviseur (Père)** : Le processus père lit en boucle les entiers arrivant dans le tube. À chaque réception, il incrémente une variable globale entière `total_traite`. Dès que le tube est fermé par tous les Workers (détection de la fin de fichier EOF), le superviseur arrête le thread de monitoring et affiche le bilan final.
4. **Monitoring (Thread)** : Le thread de monitoring s'exécute en tâche de fond dans le Superviseur. Toutes les secondes, il accède à la variable globale `total_traite` pour afficher un message d'avancement sur la sortie standard.
5. **Synchronisation** : L'accès à la variable globale `total_traite` partagée entre le thread de monitoring et le processus père doit être rigoureusement protégé par un mutex.
6. **Gestion de l'arrêt d'urgence (Signaux)** : Si l'utilisateur tape `Ctrl+C` dans le terminal, le Superviseur doit intercepter ce signal (`SIGINT`), envoyer un signal `SIGTERM` à chacun de ses Workers encore actifs, attendre la mort propre de tous ses fils pour éviter les processus zombies, puis quitter.